

DAY 6

Python Tuples, Sets and Dictionaries

Muhammad Ahmad
60-Day Intensive Data Science Learning Journey
Goal: Data Scientist

Focus: Tuples, sets, dictionaries, structured records, filtering, basic data analysis, practice tasks, and a Student Performance Analyzer project.

1. Tuples

Tuples are ordered collections that are immutable. Existing tuple items cannot be changed.

```
student = ("Ahmad", 22, 3.23)
print(student)
```

Indexing, negative indexing and slicing

```
student = ("Ahmad", 22, 3.23)
print(student[0])
print(student[-1])

numbers = (10, 20, 30, 40, 50)
print(numbers[1:4])
```

Conversion, methods and unpacking

```
numbers = [10, 20, 30]
print(tuple(numbers))

numbers = (10, 20, 30)
print(list(numbers))

values = (10, 20, 10, 30, 10)
print(values.count(10))
print(values.index(30))

student = ("Ahmad", 22, 3.23)
name, age, cgpa = student
print(name, age, cgpa)
```

Changing a tuple item, such as `numbers[0] = 100`, raises `TypeError` because tuples are immutable.

2. Sets

Sets store unique values. Duplicate values are automatically removed. Sets do not support normal indexing.

```
numbers = {10, 20, 20, 30, 30, 40}
print(numbers)

courses = {"Python", "SQL"}
courses.add("Statistics")
courses.update(["Machine Learning", "Data Science"])
courses.discard("Java")
print(courses)
```

Set operations

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print("Union:", a.union(b))
print("Intersection:", a.intersection(b))
print("Difference:", a.difference(b))
print("Symmetric:", a.symmetric_difference(b))
```

3. Dictionaries

Dictionaries store key-value pairs and are ideal for structured records.

```
student = {"name": "Ahmad", "age": 22, "cgpa": 3.23}
```

```

print(student["name"])
print(student.get("cgpa"))

student["semester"] = 7
student["cgpa"] = 3.23
student.update({"city": "Burewala"})
student.pop("age")

for key, value in student.items():
    print(key, ":", value)

```

Nested dictionaries

```

student = {
    "name": "Ahmad",
    "academic": {"cgpa": 3.23, "semester": 7}
}
print(student["academic"]["cgpa"])

```

List of dictionaries

```

students = [
    {"name": "Ahmad", "marks": 85},
    {"name": "Ali", "marks": 72},
    {"name": "Usman", "marks": 45}
]

passed = [s for s in students if s["marks"] >= 50]
print(passed)

highest = max(students, key=lambda s: s["marks"])
print(highest)

```

Basic data analysis

```

marks = [s["marks"] for s in students]
print("Average:", sum(marks) / len(marks))
print("Highest:", max(marks))
print("Lowest:", min(marks))

```

4. Data Structure Comparison

Structure	Property	Use
List	Ordered, mutable, duplicates	Changing ordered data
Tuple	Ordered, immutable	Fixed data
Set	Unique values	Duplicates and set operations
Dictionary	Key-value pairs	Structured records

Data Science connection: These structures help represent and process raw structured data before moving to NumPy, Pandas, and larger datasets.

5. Practice Tasks with Solutions

Try each task yourself first. The solved versions are included here for revision.

Task 1: Tuple basics

```

numbers = (10, 20, 30, 40, 50)
print("First:", numbers[0])
print("Last:", numbers[-1])
print("Length:", len(numbers))

```

Task 2: List to tuple

```

numbers = [10, 20, 30, 40]
numbers_tuple = tuple(numbers)
print(numbers_tuple)

```

Task 3: Remove duplicates

```

numbers = [10, 20, 20, 30, 30, 40, 40]
unique_numbers = set(numbers)
print(unique_numbers)

```

Task 4: Set operations

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print("Union:", a.union(b))
print("Intersection:", a.intersection(b))
print("Difference:", a.difference(b))
```

Task 5: Student dictionary

```
student = {"name": "Ahmad", "age": 22, "semester": 7, "cgpa": 3.10}
student["cgpa"] = 3.23
print(student)
```

Task 6: Five students

```
students = [
    {"name": "Ahmad", "marks": 85},
    {"name": "Ali", "marks": 72},
    {"name": "Usman", "marks": 65},
    {"name": "Hamza", "marks": 91},
    {"name": "Bilal", "marks": 58}
]
for student in students:
    print(student["name"], student["marks"])
```

Task 7: Marks >= 70

```
high_scorers = [s for s in students if s["marks"] >= 70]
print(high_scorers)
```

Task 8: Average, highest and lowest

```
marks = [s["marks"] for s in students]
print("Average:", sum(marks) / len(marks))
print("Highest:", max(marks))
print("Lowest:", min(marks))
```

6. Mini Project: Student Performance Analyzer

```
students = [
    {"name": "Ahmad", "marks": [85, 78, 92]},
    {"name": "Ali", "marks": [72, 65, 80]},
    {"name": "Usman", "marks": [45, 52, 40]},
    {"name": "Hamza", "marks": [91, 88, 95]},
    {"name": "Bilal", "marks": [58, 62, 55]}
]
```

Analyzer function

```
def analyze_student(student):
    name = student["name"]
    marks = student["marks"]
    total = sum(marks)
    average = total / len(marks)
    highest = max(marks)
    lowest = min(marks)

    if average >= 80:
        grade = "A"
    elif average >= 70:
        grade = "B"
    elif average >= 60:
        grade = "C"
    elif average >= 50:
        grade = "D"
    else:
        grade = "F"

    status = "Pass" if average >= 50 else "Fail"

    return {
        "name": name, "total": total,
        "average": average, "highest": highest,
        "lowest": lowest, "grade": grade,
        "status": status
    }
```

Run the analyzer

```
for student in students:
```

```
result = analyze_student(student)
print("Name:", result["name"])
print("Total:", result["total"])
print("Average:", round(result["average"], 2))
print("Highest:", result["highest"])
print("Lowest:", result["lowest"])
print("Grade:", result["grade"])
print("Status:", result["status"])
print("-" * 30)
```

Expected grades: Ahmad A, Ali B, Usman F, Hamza A, Bilal D. Pass means average is at least 50.

7. Quick Revision

- List = ordered and mutable.
- Tuple = ordered and immutable.
- Set = unique values.
- Dictionary = key-value data.
- union() combines unique set values.
- intersection() finds common values.
- difference() finds values in one set but not the other.
- items() returns dictionary key-value pairs.
- List comprehension is useful for filtering records.
- max(..., key=...) can find the record with the highest value.

8. Day 6 Outcome

You can now select suitable Python data structures, remove duplicates, store structured records, filter data, and perform basic analysis. These skills form an important foundation for real Data Science work.

Next target: Day 7, continuing the Python foundation toward working with real data files and formats.